

QKD: Quantization-aware Knowledge Distillation

Jangho Kim^{*†}
Seoul National University
kjh91@snu.ac.kr

Yash Bhalgat^{*}
Qualcomm AI Research[‡]
Qualcomm Technologies, Inc.
ybhalgat@qti.qualcomm.com

Jinwon Lee
Qualcomm AI Research[‡]
Qualcomm Technologies, Inc.
jinwonl@qti.qualcomm.com

Chirag Patel
Qualcomm AI Research[‡]
Qualcomm Technologies, Inc.
cpatel@qti.qualcomm.com

Nojun Kwak[§]
Seoul National University
nojunk@snu.ac.kr

Abstract

Quantization and Knowledge distillation (KD) methods are widely used to reduce memory and power consumption of deep neural networks (DNNs), especially for resource-constrained edge devices. Although their combination is quite promising to meet these requirements, it may not work as desired. It is mainly because the regularization effect of KD further diminishes the already reduced representation power of a quantized model. To address this shortcoming, we propose Quantization-aware Knowledge Distillation (QKD) wherein quantization and KD are carefully coordinated in three phases. First, Self-studying (SS) phase fine-tunes a quantized low-precision student network without KD to obtain a good initialization. Second, Co-studying (CS) phase tries to train a teacher to make it more quantization-friendly and powerful than a fixed teacher. Finally, Tutoring (TU) phase transfers knowledge from the trained teacher to the student. We extensively evaluate our method on ImageNet and CIFAR-10/100 datasets and show an ablation study on networks with both standard and depthwise-separable convolutions. The proposed QKD outperformed existing state-of-the-art methods (e.g., 1.3% improvement on ResNet-18 with W4A4, 2.6% on MobileNetV2 with W4A4). Additionally, QKD could recover the full-precision accuracy at as low as W3A3 quantization on ResNet and W6A6 quantization on MobilenetV2. (See Fig. 1)

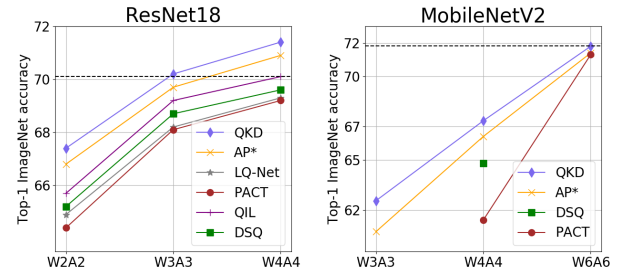


Figure 1: Top-1 accuracy on ImageNet with ResNet-18 and MobileNetV2. Our QKD is compared with various methods such as AP* [30], LQ-Net [50], PACT [6], QIL [18] and DSQ [10]. Dotted line indicates the full-Precision accuracy. More details are in Table 3 and 4.

1. Introduction

Deploying complex DNNs on resource-constrained edge devices such as smartphones or IoT devices is still challenging due to their tight memory and computation requirements. To address this, several research works have been proposed which can be roughly categorized into three folds: weight pruning [12, 11, 27, 9], quantization, [23, 50, 26, 47, 5] and knowledge distillation [15, 20, 48].

Today, a majority of edge devices require fixed-point inference for compute- and power-efficiency. Hence, quantizing the weights and activations of deep networks to a certain bit-width becomes a necessity to make them compatible to edge devices. Recently, hardware accelerators like NPUs (Neural Processing Units), NVIDIA’s Tensor Core units and even CIM (Compute-in-memory) devices have targeted support for sub-byte level processing such as 4-bit $\times$ 4-bit or 2-bit $\times$ 2-bit matrix multiply-and-accumulate operations

^{*}Equal contribution

[†]Work done while interning at Qualcomm AI Research.

[‡]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

[§]Currently a Visiting Researcher at Qualcomm Technologies, Inc.

[28, 33, 40, 42]. These are much more power- and compute-efficient than conventional 8-bit processing. Thus, there are increased demands for lower-bit quantization.

The accuracy of very low-bit quantized networks such as using 4-, 3- or 2-bits inevitably decreases because of their low representation power compared to typical 8-bit quantization. Some works [30, 34] used knowledge distillation (KD) to improve the accuracy of quantized networks by transferring knowledge from a full-precision teacher network to a quantized student network. However, even though KD is one of the widely used techniques to enhance a DNN's accuracy [15], directly applying KD to very low-bit quantized networks incurs several challenges:

1. Due to its limited representation power, quantized networks generally show lower training and test accuracies compared to the networks with full precision. On the other hand, because KD uses not only the ground truth labels but also the teacher's estimated class probability distributions, it acts as a heavy regularizer [15]. Combining these contradictory characteristics of quantization and KD could sometimes result in further degradation of performance from that of quantization-only.
2. In general, it has been shown that a powerful teacher with high accuracy can teach a student better than a weaker one [54]. Also, [20, 29] show that if there is a large gap from the capacity or inherent differences between the teacher and student, it can hinder the knowledge transfer because this knowledge is unadaptable to the student. Previous works of quantization with KD [30, 34] directly applied conventional KD to quantization along with fixed teacher network, and thus they suffer from above mentioned limitations.

In this paper, we propose Quantization-aware Knowledge Distillation (QKD) to address the above mentioned challenges, especially for very low-precision quantization. QKD consists of three phases. In the first '**self-studying**' phase, instead of directly applying knowledge distillation to the quantized student network from the beginning, we first try to find a good initialization. In the second '**co-studying**' phase, we improve the teacher by using the knowledge of the student network. This phase makes the teacher more powerful and quantization-friendly (See Section 4.5), which is essential to improve accuracy. In final '**tutoring**' phase, we freeze the teacher network and transfer its knowledge to the student. This phase saves unnecessary training time and memory of the teacher network which tends to have already saturated in the co-studying phase. The overall process of QKD is depicted in the Figure 2. Our key contributions are the following:

- We propose Quantization-aware Knowledge Distillation which can be effectively applied to very low-bit

(2-,3-,4-bit) quantized networks. Considering the characteristics of low bit networks and distillation methods, we design a combination of three phases for QKD that overcome the shortcomings posed by conventional quantization + KD methods as described above.

- We empirically verify that QKD works well on depth-wise convolution networks which are known to be difficult to quantize and our work is the first to apply KD to train low-bit depth-wise convolution networks (viz. MobileNetV2 and EfficientNet)
- We show that our QKD obtains state-of-the-art accuracies on CIFAR and ImageNet datasets compared to other existing methods. (See Figure 1.)

2. Related work

Quantization Reducing the precision of neural networks [7, 25, 53, 32, 43] has been studied extensively due to its computational and storage benefits. Although binary $\{-1, +1\}$ [7] or ternary $\{-1, 0, +1\}$ [25] quantizations are typically used, their methods only consider the quantization of weights. To fully utilize bit-wise operations, activation maps also should be quantized in the same way as weights. Some researches consider quantizing both weights and activation maps [16, 41, 35, 52, 46]. Binary neural networks [16] quantize both weights and activation maps as binary and compute gradients with binary values. XNOR-Net [35] also quantizes its weights and activation maps with scaling factors obtained by constrained optimization. Furthermore, HAQ [46] adaptively changes bit-width per layer by leveraging reinforcement learning and HW-awareness.

Recent works have tried to improve quantization further by learning the range of weights and activation maps [6, 18, 8, 45]. These approaches can easily outperform previous methods which do not train quantization-related parameters. PACT [6] proposes a clipping activation function using trainable a parameter which limits the maximum value of activation. LSQ [8] and TQT [17] introduce uniform quantization using trainable interval values. QIL [18] uses a trainable quantizer which performs both pruning and clipping. Our QKD leverages these trainable approaches in the baseline quantization implementation. On top of this quantization-only method, our elaborated knowledge distillation process boosts the accuracy, thereby achieving the state of art accuracy on low-bit quantization.

Knowledge Distillation KD is one of the most popular methods in model compression. It is widely used in many computer vision tasks. This framework transfers the knowledge of a teacher network to a smaller-sized student network in two ways: Offline and Online. First, offline KD uses a fixed pre-trained teacher networks and the knowledge transfer can happen in different ways. In [15], they

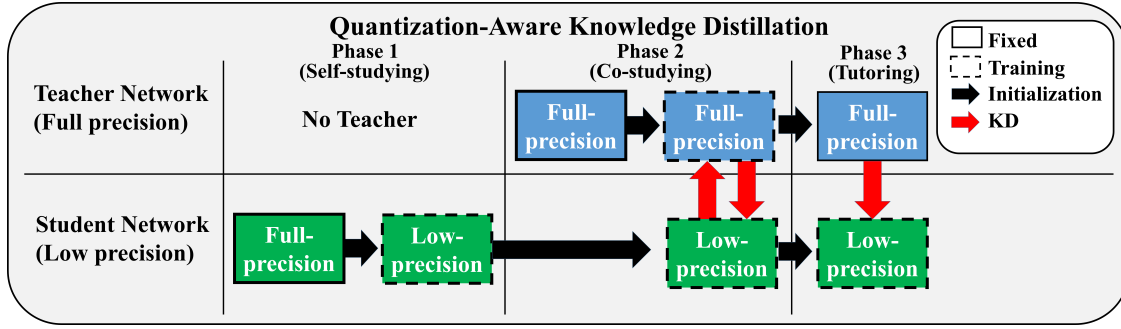


Figure 2: The overall process of QKD. Self-studying (SS) phase give a good starting point to alleviate low representative power and regularization effect of KD. Co-studying (CS) phase makes a teacher adaptable to a student and powerful than the fixed teacher. In tutoring (TU) phase, the teacher transfers its adaptable and powerful knowledge to the student.

encourage the student network to mimic the softened distribution of the teacher network. Other works [49, 14, 20, 37] transfer the information using different forms of activation maps from the teacher network. Second, online KD methods [54, 19, 51] train both the teacher and the student networks simultaneously without pre-trained teacher models. Deep Mutual Learning (DML) [51] have tried to transfer each knowledge of independent networks using KL loss, although it is not studied in the context of quantization. Our QKD uses both online and offline KD methods sequentially.

Quantization + Knowledge distillation Some researches have tried to use distillation methods to train low precision network [30, 34]. They use a full precision network as the teacher and the low bit network as a student. In Apprentice (AP) [30], the teacher and student networks are initialized with the corresponding pre-trained full precision networks and the student is then fine-tuned using distillation. Due to AP’s initialization of the student, AP tends to get stuck in a local minimum in the case of very low-bit quantized student networks. The self-study phase of QKD directly mitigates this issue. Also, using a fixed teacher, as in [30, 34], can limit the knowledge transfer due to the inherent differences between the distributions of the full-precision teacher and low-precision student network. We tackle this problem via online co-studying (CS) and offline tutoring (TU).

3. Proposed Method

In this section, we first explain the quantization method used in QKD and then describe the proposed QKD method.

3.1. Quantization

In QKD, we use a trainable uniform quantization scheme for our baseline quantization implementation. We choose uniform quantization because of its hardware-friendly characteristics. In this work, we quantize both weights and activations. So, we introduce two trainable parameters for the interval values of each layer’s weight parameters and in-

put activations similar to [8, 17, 45]. These parameters can be trained with either the task loss or a combination of the task and distillation losses. Considering k -bit quantization for weights and activations, the weight and activation quantizers are defined as follows.

Weight Quantizer: Since weights can take positive as well as negative values, we quantize each weight to integers in the range $[-2^{k-1}, 2^{k-1} - 1]$. Before rounding, the weights are first constrained to this range using a clamping function $F_W(w) \triangleq F(w, -2^{k-1}, 2^{k-1} - 1)$ where

$$F(x, \min, \max) = \begin{cases} \max & \text{if } x > \max \\ \min & \text{if } x < \min \\ x & \text{otherwise.} \end{cases} \quad (1)$$

We use a trainable parameter I_W for the interval value of weight quantization. It is trained along with the weights of the network. We can calculate the quantization level of the input weight w using a rounding operation on F_W . The overall quantization-dequantization scheme for the weights of the network is defined as follows:

$$\hat{w} = Q_W(w) = \left\lfloor F_W\left(\frac{w}{I_W}\right) + \frac{1}{2} \right\rfloor \times I_W \quad (2)$$

where $\lfloor \cdot \rfloor$ is the flooring operation. Note that the *dequantization* step (multiplication by I_W) just brings the quantized values ($\hat{w}$) back to their original range and is commonly used when emulating the effect of quantization [3, 17, 36].

Activation Quantizer: Since most of the networks today use ReLU as the activation function, the activation values are unsigned¹. Hence, to quantize the activations, we use an quantization function with the range $[0, 2^k - 1]$. The activation quantizer Q_X can be obtained as

$$\hat{x} = Q_X(x) = \left\lfloor F_X\left(\frac{x}{I_X}\right) + \frac{1}{2} \right\rfloor \times I_X \quad (3)$$

¹EfficientNet uses Swish extensively which introduces a small proportion of negative activation values, but we can save one bit per activation by clamping these negative values at 0 using unsigned quantization

where, $F_X(x) \triangleq F(x, 0, 2^k - 1)$ and x, I_X represent activation value and the interval value of activation quantization.

Prior to training, we initialize these interval values (I_W, I_X) for every layer using the min-max values of weights and activations (from one forward pass), similar to TF-Lite [1]. These quantizers are non-differentiable, so we use a straight-through estimator (STE) [2] to backprop through the quantization layer. STE approximates the gradient $\frac{d\hat{x}}{dx}$ by 1. Thus, we can approximate the gradient of loss $\mathcal{L}$, $\frac{d\mathcal{L}}{dx}$, with $\frac{d\mathcal{L}}{d\hat{x}}$

$$\frac{d\mathcal{L}}{dx} = \frac{d\mathcal{L}}{d\hat{x}} \frac{d\hat{x}}{dx} \approx \frac{d\mathcal{L}}{d\hat{x}}. \quad (4)$$

3.2. Quantization-aware Knowledge Distillation

We will now describe the three phases of QKD. Algorithm 1 depicts the overall QKD training process.

3.2.1 Phase 1: Self-studying

Directly combining quantization and KD cannot make the most of the positive effects of KD and easily cause unexpectedly lower performance. This is because of the regularizing characteristics of KD and limited representative power of the quantized network. This can cause the quantized network to easily get trapped in a poor local minima. To mitigate this issue and to provide a good starting point for KD, we train the low-bit network for some epochs using only the task loss, i.e. the standard cross entropy loss. Such a self-studying phase provides the student with good initial parameters before KD is applied.

Our method can be compared to progressive quantization (PQ) [55] which also uses two stages and progressively quantizes weights and activation maps for a good initial point. While PQ conducts progressive quantization which uses a higher precision network as a initialization and conducts iterative update between weight and activation maps, our method directly initializes the same low bit-width for the target low-bit network because learning the interval of weights and activation maps works well without iterative and progressive training.

This strategy of parameter initialization and knowledge distillation has good synergy in terms of generalization and finding a good local minimum. More specifically, our initialization scheme helps to start with a good starting point and the distillation loss guides the student network to good local minima acting as a regularizer.

3.2.2 Phase 2: Co-studying

To make a powerful and adaptable teacher, we jointly train the teacher network (full-precision) and the student network (low-precision) in an online manner. Kullback–Leibler divergence (KL) between the student and teacher distributions

Algorithm 1 Quantization-aware Knowledge Distillation

Input: Training data;

Pre-trained FP weights for teacher model T_F ;

Pre-trained FP weights for student model S_F ;

Low-bit student model weights S_L ;

Weight interval values I_W ;

Activation interval values I_X ;

Number of epochs for each phase P_1 ; P_2 ; P_3 ;

Output: Trained low-bit student weight and interval values S_L' , I_W' and I_X'

1: **Phase 1: Self-studying**

2: Init S_L with S_F ; Init I_W, I_X using min-max values of weights and one batch of activations;

3: **for** Epoch = 1 ,..., P_1 **do**

4: Update S_L, I_W and I_X by minimizing $\mathcal{L}_{ce}^s$ (6)

5: **end for**

6: **Phase 2: Co-studying**

7: Init S_L', I_W' and I_X' with S_L, I_W and I_X

8: **for** Epoch = 1 ,..., P_2 **do**

9: Update T_F by minimizing $\mathcal{L}_{KD}^T$ (9)

10: Update S_L', I_W' and I_X' by minimizing $\mathcal{L}_{KD}^S$ (8)

11: **end for**

12: **Phase 3: Tutoring**

13: **for** Epoch = 1 ,..., P_3 **do**

14: Update S_L', I_W' and I_X' by minimizing $\mathcal{L}_{KD}^S$ (8)

15: **end for**

is used to make the teacher more powerful in terms of accuracy as well as its adaptability to the student distribution than the fixed pre-trained teacher. In this framework, teacher network is trained by softened distribution of student network and vice versa. Teacher network can be adapted to the quantized student network with KL loss by awareing the distribution of the quantized network.

Assuming that there are m classes, the cross-entropy loss for both the teacher and the student networks is obtained by firstly computing the softmax posterior with temperature $\mathcal{T}$ as follows:

$$p_i(\mathbf{z}^k; \mathcal{T}) = \frac{e^{z_i^k / \mathcal{T}}}{\sum_j^m e^{z_j^k / \mathcal{T}}}, \quad (5)$$

$$\mathcal{L}_{ce}^k = - \sum_{i=1}^m y^{(i)} \log(p_i(\mathbf{z}^k; 1)), \quad (6)$$

where $\mathbf{z}^k$ and $\mathcal{L}_{ce}^k$ represent logit and cross-entropy of the k -th network, i.e. the student or the teacher ($k = \{S, T\}$). The temperature value, $\mathcal{T}$, is used to make distribution softer for using the dark knowledge. We can compute the KL loss between student and teacher network using logits.

$$KL(z^T || z^S; \mathcal{T}) = \sum_{i=1}^m p_i(z^T; \mathcal{T}) \log\left(\frac{p_i(z^T; \mathcal{T})}{p_i(z^S; \mathcal{T})}\right). \quad (7)$$

Then, we update each network with cross entropy and KL loss as below:

$$\mathcal{L}_{KD}^S = \mathcal{L}_{ce}^S + \mathcal{T}^2 \times KL(z^T || z^S; \mathcal{T}) \quad (8)$$

$$\mathcal{L}_{KD}^T = \mathcal{L}_{ce}^T + \mathcal{T}^2 \times KL(z^S || z^T; \mathcal{T}) \quad (9)$$

$\mathcal{L}_{KD}^S$ and $\mathcal{L}_{KD}^T$ refer to the loss of student and teacher network, respectively. We multiply $\mathcal{T}^2$ because the gradient scale of logits decrease as much as $1/\mathcal{T}^2$.

3.2.3 Phase 3: Tutoring

After a few epochs of co-studying, the accuracy of teacher network starts saturating. This is because the teacher (full-precision) has relatively high representative power than the student (low-precision). To reduce the computational cost and memory in calculating the gradient of the teacher network, we freeze the teacher network and train only the low-bit student network with $\mathcal{L}_{KD}^S$ loss in an offline manner. In this phase, we use the knowledge of a teacher network which is now more quantization-aware as a result of co-studying. As we will show later (See Section 4.5), using tutoring gives us equal or better student performance compared to only using co-studying throughout the training.

4. Experiments

We perform a comprehensive evaluation of our method on the CIFAR10/100 [24] and ImageNet [38] datasets. We compare the proposed QKD with existing state-of-the-art quantization methods on 2, 3, and 4-bits (i.e., W2A2, W3A3, W4A4). To show the robustness of our method, we provide comparisons on standard convolutions (i.e., ResNet [13]) as well as depth-wise separable convolutions (i.e., MobileNetV2 [39] and EfficientNet [44]). Furthermore, we perform an extensive ablation study to analyze the effectiveness of the different components of QKD. Following methods are considered for performance comparison:

1. **‘Baseline (BL)’** is the baseline quantization-only version of QKD as described in 3.1; no teacher is used during training. We train the low precision network using cross-entropy and initialize the weights with the pre-trained full-precision weights.
2. **‘SS + BL’** is BL, but the low-bit network is initialized with the weights and interval values trained in the self-studying (SS) phase.²
3. **‘AP*’** is a modified version of the original Apprentice [30] method for knowledge distillation. The original Apprentice uses WRPN scheme [31]. We replace this quantizer with BL and initialize both teacher and student with pre-trained full-precision network.

²Note that SS is the same as BL, only difference being that during SS, the low-bit network is trained for much fewer epochs.

4. **‘SS + AP*’** is AP* initialized with the weights and interval values trained in the self-studying (SS) phase for the low-precision network.
5. **‘CS + TU’** means that we initialize the student network with pre-trained full-precision network the same way as AP*, then perform “Co-studying” and “Tutoring” between the teacher and the student using BL.
6. **‘QKD (SS + CS + TU)’** is our proposed method. We initialize the student network by the SS phase. Then, we perform KD in the CS + TU phase.

4.1. Implementation Details

We quantize the weights and input activations of all the Conv and Linear layers as described in Section 3.1. We quantize first and last layer to 8-bits to ensure compatibility on any fixed-point hardware. We set the temperature value $\mathcal{T}$ as 2 in QKD. In all the experiments, we use the same settings while training all 6 baselines mentioned above.

CIFAR We train for up to 200 epochs with step learning rate schedule with SGD same as [50]. We use the starting learning rate (LR) of 0.1 and 0.01 for models corresponding to CIFAR-10/100 respectively. We use 30 epochs for SS phase by dividing LR with 10 for every 10 epoch. After SS phase, we reset the LR to 0.1 and 0.01 corresponding to CIFAR-10/100. Then, we use other 170 epochs for CS + TU phase. LR learning rate is divided by 10 at 80 and 120 epoch. We use 100 epochs for CS and the remaining 70 epochs for TU. For teacher network, we use the same schedule as the student for the CS phase, but we freeze the teacher in TU. Compared to the LR used for student model’s weight parameters, we use 100 times smaller LR for updating I_W and same LR for updating I_X .

ImageNet For all our compared methods, we used total 120 epochs for training (this is the same length as QIL [18]). In the methods SS + BL, SS + AP* and SS + CS + TU, we finetune the student model for 50 epochs in the SS phase and the remaining 70 epochs are used for the rest. In the CS + TU method, the first 60 epochs are for CS and the remaining 60 epochs for TU. For training the student, we used a Mixed Optimizer (MO) setting [22]. In the MO setting, we use SGD [4] optimizer to update the weights and Adam [21] for updating the interval values (I_W , I_X). We found that the exponential learning rate schedule works best for QKD. For the ResNet architectures, we use an initial learning rate of 0.001 for both SS and CS + TU phases. For EfficientNet and MobileNetV2, initial learning rate of 0.003 was used. Similar to CIFAR-100 setting, we use 100 times smaller LR for I_W and same LR for I_X , compared to LR for model weights. For I_W and I_X , not much fine-tuning was required because of the Mixed Optimizer setting. More details are described in supplementary details.

Table 1: CIFAR-10 top-1 accuracy

Network	Method	W2A2	W3A3	W4A4
ResNet-20 (FP : 91.6)	DoReFa-Net [52]	88.2	89.9	90.5
	PACT [6]	89.7	91.1	91.3
	LQ-Net [50]	90.2	91.6	–
	BL	89.9	91.8	92.1
	SS + BL	90.1	91.9	92.3
	AP*	88.6	91.8	92.2
	SS + AP*	90.2	92.3	92.3
	CS + TU	88.5	92.1	92.5
	QKD	90.5	92.7	93.1

4.2. CIFAR-10 Results

We evaluate our method on the CIFAR-10 dataset. The performance numbers are shown in the Table 1. We use ResNet-56 (FP : 93.4%) as a teacher network for both AP* and QKD. Interestingly, for AP* and CS + TU, which combine BL and knowledge distillation, performance is worse than BL at W2A2. This is because the network has low-representative power at W2A2 and can be negatively affected by the heavy regularization imposed by KD. To alleviate this issue, we use the weights trained in the SS phase for a better initialization for the weights of the student. SS + AP* and QKD outperform the existing methods and the Baseline (BL) method for all the cases. These results suggest that initialization is very important in combining quantization and *any* KD method. In our proposed method, 2% gain was observed at W2A2 with the SS phase. Compared to the distillation method AP*, QKD which also trains the teacher shows a significant improvement in performance. QKD also provides best results for W3A3 and W4A4.

4.3. CIFAR-100 Results

Table 2 shows the experimental results on CIFAR-100. We use a ResNet-56 (FP : 73.4%) as a teacher in this experiment. These experiments show similar tendencies to those with CIFAR-10. The accuracy increases with the SS phase which helps us start with a better initialization especially in very low-bit quantization (W2A2). Considering KD methods, QKD performs significantly better than SS+AP*. We attribute this gain in accuracy to the use of an adaptable teacher trained during co-studying compared to a fixed teacher. Further discussion on this is provided in Section 4.5. Compared to the full-precision version of ResNet-32, the W3A3 and W4A4 quantized versions have a 1.4% and 2.5% gain respectively.

4.4. ImageNet Results

4.4.1 Standard Convolutions

Table 3 shows the performance of the proposed method on the original ResNet18, ResNet34 and ResNet50 architectures [13]. We compare our proposed QKD method with

Table 2: CIFAR-100 top-1 accuracy

Network	Method	W2A2	W3A3	W4A4
ResNet-32 (FP : 70.8)	BL	65.4	69.8	70.9
	SS + BL	65.7	70.5	70.8
	AP*	63.5	70.3	71.5
	SS + AP*	66.1	70.6	71.6
	CS + TU	62.3	71.8	73.2
	QKD	66.4	72.2	73.3

QIL [18], PACT [6], DSQ [10] and LQ-Nets [50], which are current SOTA methods that show results with 2-, 3-, and 4-bit quantization on the original ResNet architectures. We use ResNet101, ResNet50 and ResNet34 as teachers for the student networks ResNet50, ResNet34 and ResNet18 respectively.

We observed that the QKD method outperforms the existing approaches in terms of both top-1 and top-5 accuracy. Our distillation method consistently gave us 0.5-1.1% gain in top-1 accuracy across all bit-widths over our baseline quantization method (BL). QKD even exceeds the full-precision accuracy by more than 1% at W4A4 for all ResNet architectures. Interestingly, CS+TU outperforms AP* everywhere but SS+AP* has better performance than CS+TU at W2A2. This again confirms the efficacy of using self-studying especially at 2-bit quantization. Also, it can be seen that QKD (SS+CS+TU) outperforms all the other methods at W2A2.

4.4.2 Depthwise Separable Convolutions

Table 4 shows the performance of our method on MobileNetV2 (width_multiplier=1.0) [39] and EfficientNet-B0 [44]. DSQ [10] shows results on MobileNetV2 at W4A4. We refer to the PACT [6] performance on MobileNetV2 from the HAQ paper [46]. EfficientNet-B0 is the current state-of-the-art on ImageNet among architectures with similar parameter and MAC count. Hence, we also provide the results of our method on EfficientNet-B0. We include W6A6 into our set of bit-widths. We use MobileNetV2 (width=1.4) as teacher for the student network MobileNetV2 (width=1.0) and EfficientNet-B1 as the teacher for EfficientNet-B0.

In general, we see higher gains with the QKD (over the BL and AP* methods) with both MobileNetV2 and EfficientNet-B0 than what we observed on the ResNet architectures. If we compare a similar accuracy range (66~67%) which is observed with W4A4 on MobileNetV2 and W3A3 on EfficientNet-B0, we can see 1.2% and 1.7% top-1 accuracy improvement respectively with QKD over BL. Interestingly, we observe a significant difference of 1.3% between AP* and SS+AP* at W3A3 with MobileNetV2.

Table 3: ImageNet performance comparison on architectures with standard convolutions

Bitwidth	Method	ResNet18		ResNet34		ResNet50	
		top-1	top-5	top-1	top-5	top-1	top-5
W32A32	-	70.1	89.5	73.5	91.7	76.3	93.0
W4A4	PACT [6]	69.2	89.0	-	-	76.5	93.2
	LQ-Nets [50]	69.3	88.8	-	-	75.1	92.4
	DSQ [10]	69.6	-	72.8	-	-	-
	QIL [18]	70.1	-	73.7	-	-	-
	BL	70.7	89.4	73.8	91.5	76.6	93.2
	SS + BL	70.8	89.4	74.9	91.5	76.6	93.2
	AP*	70.9	89.5	74.0	91.7	76.7	93.2
	SS + AP*	71.1	89.6	74.2	91.8	76.9	93.3
	CS + TU	71.1	90.1	74.4	91.9	77.0	93.5
	QKD	71.4	90.3	74.6	92.1	77.3	93.6
W3A3	PACT [6]	68.1	88.2	-	-	75.3	92.6
	LQ-Nets [50]	68.2	87.9	71.9	90.2	74.2	91.6
	DSQ [10]	68.7	-	72.5	-	-	-
	QIL [18]	69.2	-	73.1	-	-	-
	BL	69.4	88.5	73.3	90.5	75.3	92.6
	SS + BL	69.5	88.5	73.3	90.6	75.4	92.7
	AP*	69.7	88.7	73.5	90.7	76.0	92.7
	SS + AP*	69.9	88.7	73.7	90.8	76.1	92.8
	CS + TU	70.0	89.4	73.7	91.2	76.2	92.9
	QKD	70.2	89.9	73.9	91.4	76.4	93.2
W2A2	PACT [6]	64.4	85.6	-	-	72.2	90.5
	LQ-Nets [50]	64.9	85.9	69.8	89.1	71.5	90.3
	DSQ [10]	65.2	-	70.0	-	-	-
	QIL [18]	65.7	-	70.6	-	-	-
	BL	66.7	86.7	71.0	89.8	73.3	91.0
	SS + BL	66.8	86.9	71.0	89.9	73.3	91.0
	AP*	66.8	86.9	71.1	89.9	73.4	91.0
	SS + AP*	67.2	87.3	71.3	90.2	73.7	91.2
	CS + TU	67.1	87.2	71.3	90.1	73.6	91.2
	QKD	67.4	87.5	71.6	90.3	73.9	91.6

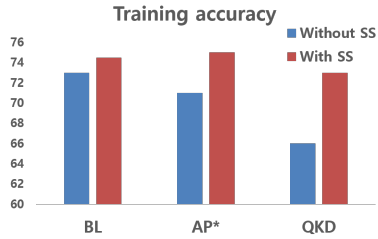


Figure 3: Training accuracy of student network using BL, AP* and QKD at W2A2 on CIFAR-100. Red and blue represent the accuracy with and without Self-studying (SS)

With W2A2, we observe a drastic drop in top-1 accuracy. This was expected since depth-wise convolution layers are known to be highly sensitive to quantization [46, 17]. So, we ran another set of experiments where the weights and input activations of the depth-wise separable layers are quantized to W8A8 whereas the rest of the layers are quantized to W2A2, W3A3 or W4A4. We include these results in the supplementary details.

Table 4: ImageNet performance comparison on architectures with depthwise separable convolutions

Bitwidth	Method	MobileNetV2		EfficientNet-B0	
		top-1	top-5	top-1	top-5
W32A32	-	71.8	90.3	76.3	93.2
W6A6	PACT [6]	71.3	90.0	-	-
	BL	71.2	89.9	74.5	92.3
	SS + BL	71.3	90.0	74.6	92.3
	AP*	71.4	90.0	74.8	92.4
	SS + AP*	71.4	90.0	75.0	92.4
	CS + TU	71.5	90.1	75.1	92.6
	QKD	71.8	90.2	75.4	92.6
W4A4	PACT [6]	61.4	83.7	-	-
	DSQ [10]	64.8	-	-	-
	BL	66.1	86.2	71.9	90.4
	SS + BL	66.3	86.3	72.0	90.4
	AP*	66.4	86.4	72.3	90.6
	SS + AP*	66.6	86.5	72.6	90.7
	CS + TU	66.7	86.7	72.8	91.0
	QKD	67.4	87.0	73.1	91.2
W3A3	BL	60.1	83.0	67.5	85.8
	SS + BL	61.5	83.4	67.7	85.9
	AP*	60.7	83.2	68.4	86.1
	SS + AP*	62.0	83.5	68.7	86.2
	CS + TU	62.0	83.5	68.8	86.4
	QKD	62.6	84.0	69.2	86.9
W2A2	BL	37.7	64.1	43.5	67.5
	SS+BL	42.7	66.2	44.9	67.6
	AP*	39.8	65.3	46.4	68.2
	SS+AP*	43.9	67.3	47.7	68.5
	CS + TU	43.6	66.9	48.0	68.7
	QKD	45.7	68.1	50.0	69.3

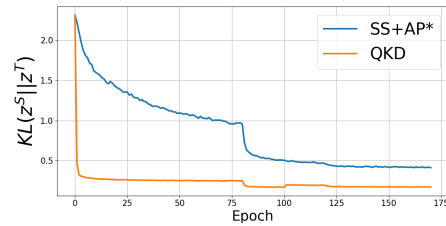


Figure 4: KL-divergence between teacher & student distributions on CIFAR100 with QKD and SS+AP* (SS part is not shown in both because it doesn't involve the teacher network).

4.5. Discussion

Effectiveness of Self-studying Figure 3 shows the training accuracy of the student in the final epoch with and without the Self-Studying phase on the CIFAR-100 dataset. At very low-bit quantization, the model has low representative power and without SS, it gets stuck in a local minima as is reflected from the lower training accuracy. The SS phase helps the student start from a better initialization point and KD then guides it to a better local minima, hence increasing the training accuracy. Note that although training accuracy

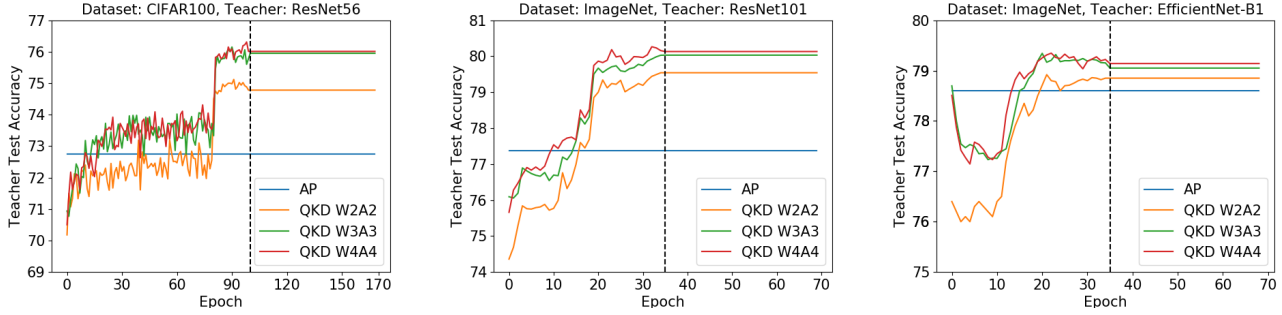


Figure 5: Accuracy of the teacher network during the CS + TU phase for 3 different scenarios. The vertical dotted line separates Co-studying and Tutoring phases. The blue horizontal line shows the teacher accuracy for Apprentice (AP*) method.

of QKD seems quite lower than others, its test accuracy is better than others, as was shown in Table 2. This tendency is usually seen when using KD in general full-precision domain because KD regularizes the network for enhanced generalization performance. In QKD, the gap between blue and red one is large compared to BL, meaning that it suffers from regularization effect more.

Adaptability of teacher network In Co-studying, we train the teacher network with $\mathcal{L}_{KD}^T$ (9) with the goal of making it’s distribution more adaptable to that of the low-precision student. In Figure 4, we plot the KL divergence between the teacher and student class distributions during QKD training and during SS+AP* training for one of the CIFAR-100 experiments. The KL-divergence during QKD is consistently lower than that of SS + AP*. This indicates that our QKD makes the teacher more adaptable to the low-precision student in terms of the similarity of their class distribution.

Powerful teacher network In addition to improving teacher’s adaptability to the quantized student network, we observed that the teacher network’s accuracy significantly increases during the Co-studying phase. This can be attributed to the regularizing effect and the knowledge that is being transferred from the student’s posterior distribution. Figure 5 shows the variation of teacher accuracy with epochs for different settings during the CS + TU phase, compared to a fixed teacher used in AP*. The improvement in teacher accuracy was most in the case of W4A4 and the least with W2A2. The reason could be that knowledge transferred by a W2A2 quantized network is limited.

Reasoning behind Tutoring phase From Figure 5, we note that the teacher accuracy saturates towards the end of co-studying. This is due to the higher representation power of the teacher. Considering this, we freeze the teacher and turn into the tutoring phase. This helps tremendously in terms of training speed since now only the student is being trained. Interestingly, we also see performance gains by using SS + CS + TU (50 + 35 + 35 epochs) instead of just SS

Table 5: CIFAR-100 performance

Network	Method	W2A2	W3A3	W4A4
ResNet-32	AD	62.1	70.2	72.2
	SS + AD	63.2	70.4	72.3
	QKD	66.4	72.2	73.3

+ CS (50 + 70 epochs). To verify this, we used SS + CS to train ResNet18 at 2-bit and 4-bit with ResNet34 as teacher. The performance of SS + CS was 0.2% lower in case of W4A4 and 0.5% lower with W2A2 compared to QKD.

Activations vs. class posterior The works [49, 20, 14, 37] have shown promising performance in the full-precision (FP) domain (teacher and student are both FP) by using offline activation (feature map) distillation. To compare distilling activations and the softmaxed posterior used in our QKD in terms of effectiveness on training a quantized network, we transfer the activations of the last layer of the teacher (full-precision) using L2 loss to the student (low-precision) similar to [20, 37]. We use a simple regressor used in [37, 14]. We will call this baseline as activation distillation (AD). We use ResNet-56 as a teacher. Table 5 shows comparison between AD and QKD. AD has reasonable performance at 4bits but it loses power when we use lower bit-widths. The reason is that the activation distributions between low-precision and full-precision are quite different so it would not be a good guidance for the student. Hence, we verify that using posterior is more useful than activations in terms of training a low-precision network.

5. Conclusion

In this paper, we propose a Quantization-aware Knowledge Distillation method that can be effectively applied to very low-bit quantization. We propose a combination of self-studying, co-studying and tutoring methods to effectively combine model quantization and KD, wherein we provide a comprehensive ablation study of the impact of

each of these methods on the quantized model accuracy. We show how self-studying is important in alleviating the regularization effect imposed by KD and how co-studying enhances the quantization performance by making the teacher more adaptable and more powerful in terms of accuracy. Overall, with an extensive set of experiments, we show that QKD gives significant performance boost over our baseline quantization-only method and outperforms the existing state-of-the-art approaches. We demonstrate QKD's results on networks with both standard and depthwise separable convolutions and show that we can recover the full-precision accuracy at as low as W3A3 quantization of the ResNet architectures and W6A6 quantization of MobileNetV2.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org. **4**
- [2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. **4**
- [3] Aishwarya Bhandare, Vamsi Sripathi, Deepthi Karkada, Vivek Menon, Sun Choi, Kushal Datta, and Vikram Sale-tore. Efficient 8-bit quantization of transformer neural machine language translation model. *arXiv preprint arXiv:1906.00532*, 2019. **3**
- [4] Léon Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186. Springer, 2010. **5**
- [5] Zhaowei Cai, Xiaodong He, Jian Sun, and Nuno Vasconcelos. Deep learning with low precision by half-wave gaussian quantization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5918–5926, 2017. **1**
- [6] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018. **1, 2, 6, 7**
- [7] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. Binaryconnect: Training deep neural networks with binary weights during propagations. In *Advances in neural information processing systems*, pages 3123–3131, 2015. **2**
- [8] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization, 2019. **2, 3**
- [9] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018. **1**
- [10] Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. Differentiable soft quantization: Bridging full-precision and low-bit neural networks, 2019. **1, 6, 7**
- [11] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015. **1**
- [12] Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. In *Advances in neural information processing systems*, pages 1135–1143, 2015. **1**
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. **5, 6**
- [14] Byeongho Heo, Jeesoo Kim, Sangdoo Yun, Hyojin Park, Nojun Kwak, and Jin Young Choi. A comprehensive overhaul of feature distillation. *arXiv preprint arXiv:1904.01866*, 2019. **3, 8**
- [15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. **1, 2**
- [16] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. In *Advances in neural information processing systems*, pages 4107–4115, 2016. **2**
- [17] Sambhav R Jain, Albert Gural, Michael Wu, and Chris H Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019. **2, 3, 7**
- [18] Sangil Jung, Changyong Son, Seohyung Lee, Jinwoo Son, Jae-Joon Han, Youngjun Kwak, Sung Ju Hwang, and Changkyu Choi. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4350–4359, 2019. **1, 2, 5, 6, 7**
- [19] Jangho Kim, Minsung Hyun, Inseop Chung, and Nojun Kwak. Feature fusion for online mutual knowledge distillation. *arXiv preprint arXiv:1904.09058*, 2019. **3**
- [20] Jangho Kim, SeongUk Park, and Nojun Kwak. Paraphrasing complex network: Network compression via factor transfer. In *Advances in Neural Information Processing Systems*, pages 2760–2769, 2018. **1, 2, 3, 8**
- [21] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. **5**
- [22] Guillaume Klein, Yoon Kim, Yuntian Deng, Jean Senellart, and Alexander M. Rush. OpenNMT: Open-source toolkit for neural machine translation. In *Proc. ACL*, 2017. **5**

- [23] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018. **1**
- [24] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Technical report, Cite-seer, 2009. **5**
- [25] Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016. **2**
- [26] Darryl Lin, Sachin Talathi, and Sreekanth Annapureddy. Fixed point quantization of deep convolutional networks. In *International Conference on Machine Learning*, pages 2849–2858, 2016. **1**
- [27] Zhuang Liu, Mingjie Sun, Tinghui Zhou, Gao Huang, and Trevor Darrell. Rethinking the value of network pruning. *arXiv preprint arXiv:1810.05270*, 2018. **1**
- [28] Stefan Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S Vetter. Nvidia tensor core programmability, performance & precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 522–531. IEEE, 2018. **2**
- [29] Seyed-Iman Mirzadeh, Mehrdad Farajtabar, Ang Li, and Hassan Ghasemzadeh. Improved knowledge distillation via teacher assistant: Bridging the gap between student and teacher. *arXiv preprint arXiv:1902.03393*, 2019. **2**
- [30] Asit Mishra and Debbie Marr. Apprentice: Using knowledge distillation techniques to improve low-precision network accuracy. *arXiv preprint arXiv:1711.05852*, 2017. **1, 2, 3, 5**
- [31] Asit Mishra, Eriko Nurvitadhi, Jeffrey J Cook, and Debbie Marr. Wrpn: wide reduced-precision networks. *arXiv preprint arXiv:1709.01134*, 2017. **5**
- [32] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. Data-free quantization through weight equalization and bias correction. *arXiv preprint arXiv:1906.04721*, 2019. **2**
- [33] Yu Pan, Peng Ouyang, Yinglin Zhao, Wang Kang, Shouyi Yin, Youguang Zhang, Weisheng Zhao, and Shaojun Wei. A multilevel cell stt-mram-based computing in-memory accelerator for binary convolutional neural network. *IEEE Transactions on Magnetics*, 54(11):1–5, 2018. **2**
- [34] Antonio Polino, Razvan Pascanu, and Dan Alistarh. Model compression via distillation and quantization. *arXiv preprint arXiv:1802.05668*, 2018. **2, 3**
- [35] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European Conference on Computer Vision*, pages 525–542. Springer, 2016. **2**
- [36] Andres Rodriguez, Eden Segal, Etay Meiri, Evarist Fomenko, Y Jim Kim, Haihao Shen, and Barukh Ziv. Lower numerical precision deep learning inference and training. *Intel White Paper*, 2018. **3**
- [37] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. Fitnets: Hints for thin deep nets. *arXiv preprint arXiv:1412.6550*, 2014. **3, 8**
- [38] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015. **5**
- [39] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4510–4520, 2018. **5, 6**
- [40] Hardik Sharma. *ACCELERATED DEEP LEARNING FOR THE EDGE-TO-CLOUD CONTINUUM: A SPECIALIZED FULL STACK DERIVED FROM ALGORITHMS*. PhD thesis, Georgia Institute of Technology, 2019. **2**
- [41] Daniel Soudry, Itay Hubara, and Ron Meir. Expectation backpropagation: Parameter-free training of multilayer neural networks with continuous or discrete weights. In *Advances in Neural Information Processing Systems*, pages 963–971, 2014. **2**
- [42] Huseyin Ekin Sumbul, Phil Knag, Gregory K Chen, Raghavan Kumar, Abhishek Sharma, Sasikanth Manipatruni, Amrita Mathuriya, Ram Krishnamurthy, and Ian A Young. Compute in memory circuits with multi-vdd arrays and/or analog multipliers, Feb. 7 2019. US Patent App. 16/147,004. **2**
- [43] Wonyong Sung, Sungho Shin, and Kyuyeon Hwang. Resiliency of deep neural networks under quantization. *arXiv preprint arXiv:1511.06488*, 2015. **2**
- [44] Mingxing Tan and Quoc V Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *arXiv preprint arXiv:1905.11946*, 2019. **5, 6**
- [45] Stefan Uhlich, Lukas Mauch, Kazuki Yoshiyama, Fabien Cardinaux, Javier Alonso Garcia, Stephen Tiedemann, Thomas Kemp, and Akira Nakamura. Differentiable quantization of deep neural networks, 2019. **2, 3**
- [46] Kuan Wang, Zhijian Liu, Yujun Lin, Ji Lin, and Song Han. Haq: Hardware-aware automated quantization with mixed precision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8612–8620, 2019. **2, 6, 7**
- [47] Peisong Wang, Qinghao Hu, Yifan Zhang, Chunjie Zhang, Yang Liu, and Jian Cheng. Two-step quantization for low-bit neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4376–4384, 2018. **1**
- [48] Junho Yim, Donggyu Joo, Jihoon Bae, and Junmo Kim. A gift from knowledge distillation: Fast optimization, network minimization and transfer learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4133–4141, 2017. **1**
- [49] Sergey Zagoruyko and Nikos Komodakis. Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer. *arXiv preprint arXiv:1612.03928*, 2016. **3, 8**
- [50] Dongqing Zhang, Jiaolong Yang, Dongqiangzi Ye, and Gang Hua. Lq-nets: Learned quantization for highly accurate and compact deep neural networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 365–382, 2018. **1, 5, 6, 7**
- [51] Ying Zhang, Tao Xiang, Timothy M Hospedales, and Huchuan Lu. Deep mutual learning. In *Proceedings of the*

- IEEE Conference on Computer Vision and Pattern Recognition*, pages 4320–4328, 2018. 3
- [52] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv preprint arXiv:1606.06160*, 2016. 2, 6
- [53] Chenzhuo Zhu, Song Han, Huizi Mao, and William J Dally. Trained ternary quantization. *arXiv preprint arXiv:1612.01064*, 2016. 2
- [54] Xiatian Zhu, Shaogang Gong, et al. Knowledge distillation by on-the-fly native ensemble. In *Advances in Neural Information Processing Systems*, pages 7517–7527, 2018. 2, 3
- [55] Bohan Zhuang, Chunhua Shen, Mingkui Tan, Lingqiao Liu, and Ian Reid. Towards effective low-bitwidth convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7920–7928, 2018. 4